Université Abdelmalek Essaadi
Faculté des Sciences et Techniques de Tanger
Département Génie Informatique
LICENCE | Développement web avancé
Back end (Python)| 2025/2026

Rapport de Projet de Fin de Module

Système Intégré de Gestion Scolaire (PreSkool)

Architecture, Conception UML, et Implémentation Logicielle
M.V.T

Vous pouvez consulter le projet PreSkool sur GitHub. :

Réalisé par :
Oussama JABIR
Zaineb Khbiaz
Aya lmasoudi

Sous l'encadrement de :
Prof. Sara AHSAIN

Année Universitaire 2025/2026

Table des matières

1 Modélisation UML, Ingénierie des Exigences et Architecture des Données 3

1.1 Introduction Architecturale au Système PreSkool 3

1.1.1 Pourquoi cette approche centralisée ? 3

1.2 Diagrammes des Cas d'Utilisation (Use Cases) et Scénarios 3

1.2.1 L'Administrateur (Admin) 3

1.2.2 L'Enseignant (Teacher) 4

1.2.3 L'Étudiant (Student) 5

1.3 Architecture des Données : Modélisation des classes 5

1.3.1 Décorticage des Cardinalités et Décisions Architecturales 6

2 L'Application home_auth : Sécurité et Authentification 8

2.1 Code Source : 'models.py' (L'Héritage Utilisateur) 8

2.2 Code Source : 'urls.py' (Logique de Routage) 9

2.3 Code Source : 'views.py' (L'intelligence Métier MVC) 10

2.4 Les Interfaces Utilisateurs : 'register.html' et 'login.html' 11

3 L'Application faculty : Structuration Pédagogique 12

3.1 Modélisation de Base de Données (models.py) 12

3.2 La Logique d'Affichage : Vues Basées sur les Classes (views.py) 13

3.3 Aiguillage et Variables (urls.py) 14

3.4 L'Interface et le Langage de Template (HTML) 14

4 La Gestion des Acteurs : Les Applications student et teacher 16

4.1 Modélisation de la Dépendance (models.py) 16

4.2 L'Intelligence Multi-Modèle : La Logique des Vues (views.py) 17

4.3 Analyse des Templates HTML de Profil et de Listing 18

5 Modules Avancés : Planification, Examens et Résultats 20

5.1 Modélisation de la Planification Temporelle (planning/models.py) 20

5.2 Modélisation Évaluative et Contraintes de Multiplicité (exam/models.py) . 21

5.3 Cybersécurité et Partitionnement Dynamique des Vues (exam/views.py) . 22

5.4 L'Hybridation Dynamique du Template (exam/result_list.html) 23

6 Architecture Frontend et Bilan d'Ingénierie 25

6.1 L'Héritage Dynamique (base.html) 25

6.2 Défis Techniques d'Ingénierie 27

6.2.1 1. L'Isolement des Dépendances (Environnements Virtuels) 27

6.2.2 2. Héritage des Migrations de Base de Données 27

6.2.3 3. Fichiers Statiques et Serveur de Production 27
6.3 Pistes d'Améliorations et Perspectives (Évolutivité) 27
6.4 Conclusion Générale 28

Chapitre 1

Modélisation UML, Ingénierie des Exigences et Architecture des Données

1.1 Introduction Architecturale au Système PreSkool

Le développement d'un système d'information de gestion scolaire, baptisé ici **PreSkool**, est un défi majeur de l'ingénierie logicielle. L'objectif n'est pas simplement de numériser des processus existants, mais de repenser l'interopérabilité entre les différentes strates hiérarchiques et pédagogiques d'un établissement.

Construit sur le framework **Django (Python)**, reconnu pour son architecture robuste M.V.T (Model-View-Template) et son ORM (Object-Relational Mapping) puissant, PreSkool vise à centraliser, sécuriser et rationaliser le flux de données.

1.1.1 Pourquoi cette approche centralisée ?

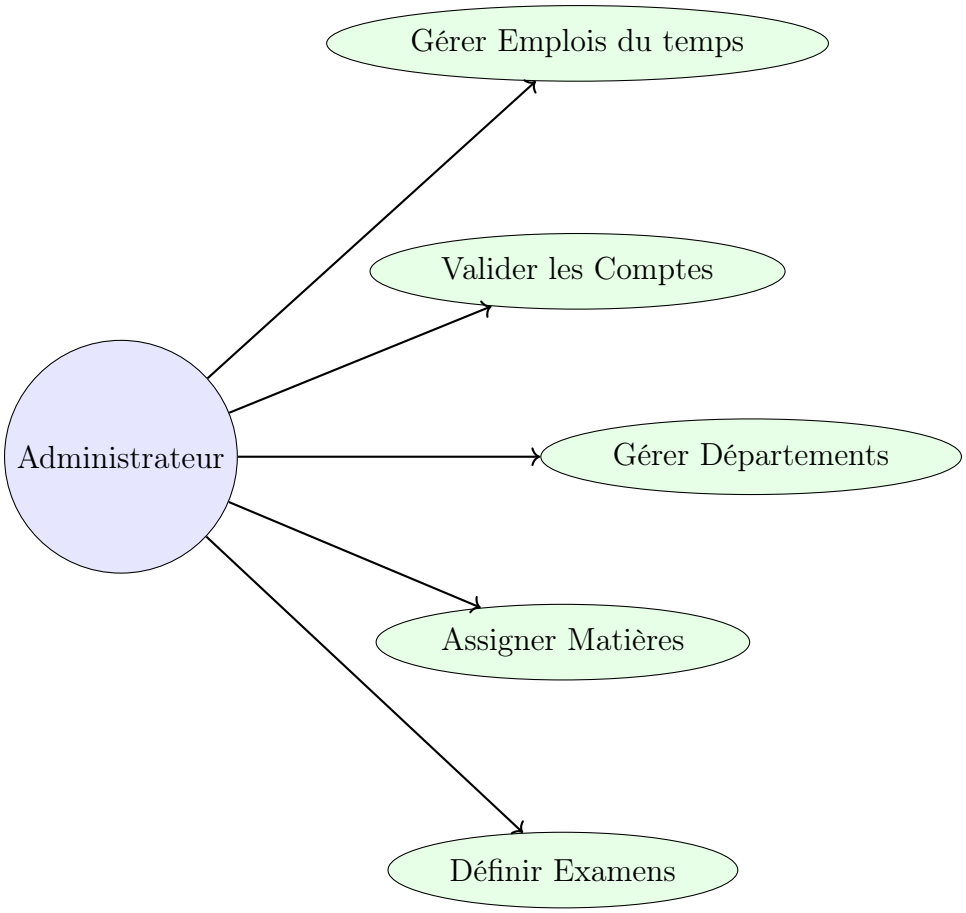
Dans les systèmes traditionnels, les données sont souvent cloisonnées en silos : un tableur pour les notes, un registre papier pour les présences, et un logiciel tiers pour la planification. Ce fractionnement mène invariablement à l'incohérence des données. Avec PreSkool, nous introduisons la notion de **Single Source of Truth** (Source Unique de Vérité). Chaque entité (un professeur, une matière, une note) n'existe qu'une seule fois en base de données de manière relationnelle. Si le professeur est mis à jour, cette modification se répercute instantanément sur tous les emplois du temps de l'école.

1.2 Diagrammes des Cas d'Utilisation (Use Cases) et Scénarios

L'ingénierie des exigences dicte que nous devons d'abord définir **qui** fait **quoi**. Le système identifie trois acteurs principaux. Voici leurs conceptions.

1.2.1 L'Administrateur (Admin)

L'Administrateur représente l'entité dirigeante de l'école. Son rôle est omnipotent mais régi par des règles de gestion strictes (CRUD).

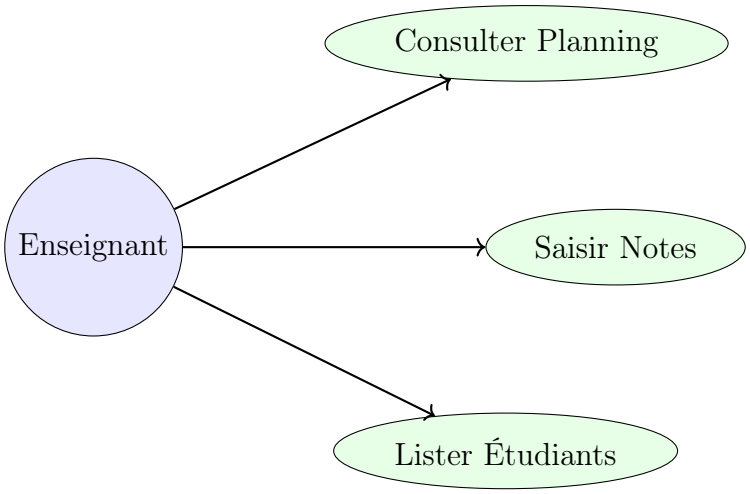


Scénario 1 : Valider les Comptes Utilisateurs
Pourquoi ? Par défaut, en cybersécurité, le principe du moindre privilège (Zero Trust) s'applique. Quiconque s'inscrit sur la plateforme est d'abord rejeté techniquement via `is_authorized = False`.
Préconditions : Un nouvel utilisateur a soumis le formulaire d'inscription.
Scénario Nominal : L'administrateur se connecte, accède au tableau de bord, identifie le compte en attente, vérifie l'identité, et bascule le booléen `is_authorized` sur `True`.
Postconditions : L'utilisateur peut désormais s'authentifier.

Scénario 2 : Gérer les Départements et Assigner les Matières
Pourquoi ? Séparer la Matière du Professeur empêche les anomalies de suppression spatio-temporelle. Si un professeur quitte l'école, on ne doit pas effacer la matière "Mathématiques".
Préconditions : L'administrateur est authentifié.
Scénario Nominal : L'admin crée un Département, puis crée une matière qu'il lie à ce département, pour enfin y assigner un professeur.
Postconditions : La matière est enregistrée avec une clé étrangère pointant vers le professeur respectif.

1.2.2 L'Enseignant (Teacher)

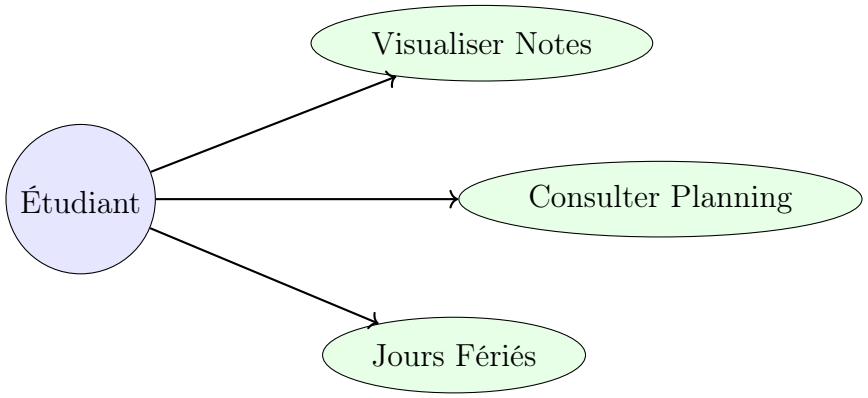
L'enseignant interagit au quotidien avec le système pour assurer le suivi pédagogique.



Scénario 1 : Saisir et Modifier des Notes
Pourquoi ? Confier la modification de l'entité Result uniquement au professeur en charge diminue le risque d'altération accidentelle des dossiers scolaires.
Préconditions : Un Examen a été créé par l'administration.
Scénario Nominal : L'enseignant clique sur "Saisir Résultat", sélectionne l'élève, et entre la note.
Postconditions : La table Result est mise à jour avec une contrainte SQL Unique qui empêche les doublons.

1.2.3 L'Étudiant (Student)

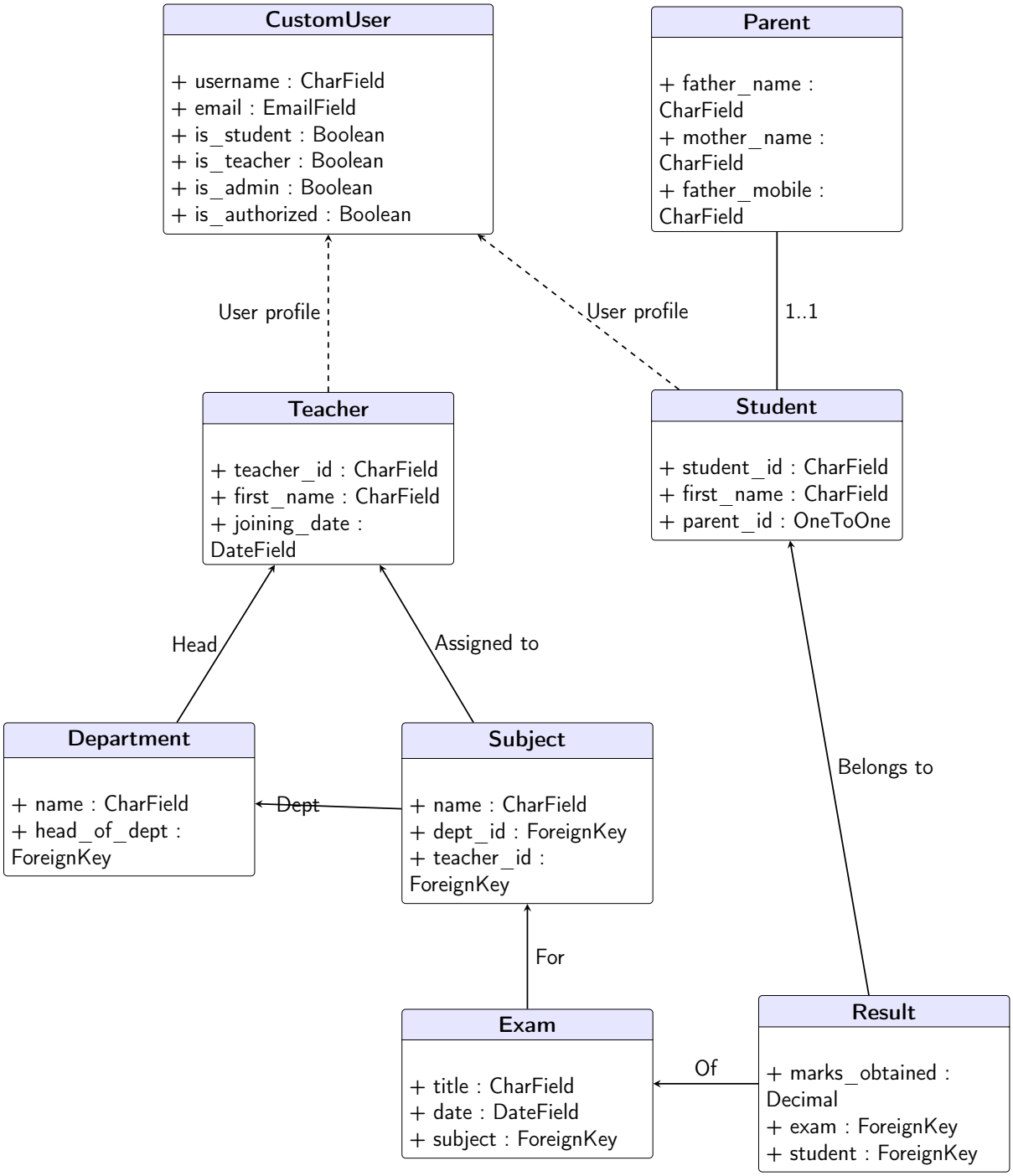
L'accès de l'étudiant est intentionnellement restreint à de la "Vue" (Read-Only).



Scénario 1 : Visualiser ses Résultats
Pourquoi ? Assurer la confidentialité via la requête de filtre strict.
Préconditions : Le compte est approuvé et des notes existent.
Scénario Nominal : L'étudiant consulte l'onglet "Mes Résultats", la base renvoie de la lecture seule.
Postconditions : Aucune modification de l'état système n'advient.

1.3 Architecture des Données : Modélisation des classes

Le diagramme de classes suivant expose l'intégralité du squelette relationnel du projet.



1.3.1 Décorticage des Cardinalités et Décisions Architecturales

La Restructuration de AbstractUser : Placer `is_student`, `is_teacher` et `is_admin` directement dans `CustomUser`. En insérant ces champs comme booléens, nous économisons des requêtes aux tables intermédiaires massives. La donnée de Rôle est déjà chargée en mémoire vive (RAM) avec la session utilisateur, accélérant le processus logique.

Relation OneToOne (Student - Parent) : La relation `OneToOne` force la symétrie absolue. Le paramètre `on_delete=models.CASCADE` démontre une "Composition" stricte en UML. Si le parent est effacé des registres, le compte enfant disparaît instantanément.

L'absence de ManyToMany dans Exam/Result : Nous avons consciemment créé l'entité matérialisée `Result` en tant que *classe-association* au lieu d'utiliser un `ManyToMany` non-explicité. L'index combiné gère parfaitement les conflits avec le système S.G.B.D.R.

Chapitre 2

L'Application home_auth : Sécurité et Authentification

Ce chapitre propose une analyse microscopique de l'application socle de PreSkool : `home_auth`. Elle orchestre le routage des requêtes HTTP de connexion, l'enregistrement des nouveaux acteurs et la sérialisation des mots de passe.

2.1 Code Source : 'models.py' (L'Héritage Utilisateur)

En Django, le modèle standard est restrictif. Nous l'avons outrepassé en construisant `CustomUser`.

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from django.core.mail import send_mail
from django.conf import settings
import uuid
from django.utils.crypto import get_random_string
from django.utils import timezone

class CustomUser(AbstractUser):
    username = models.CharField(max_length=100, unique=True)
    email = models.EmailField(max_length=255, unique=True, db_index=True
   )
    is_authorized = models.BooleanField(default=False)

    first_name = models.CharField(max_length=30, blank=True)
    last_name = models.CharField(max_length=30, blank=True)
    date_joined = models.DateTimeField(auto_now_add=True)

    is_student = models.BooleanField(default=False)
    is_teacher = models.BooleanField(default=False)
    is_admin = models.BooleanField(default=False)

    def __str__(self):
        return self.username

class PasswordResetRequest(models.Model):
    user = models.ForeignKey('CustomUser', on_delete=models.CASCADE)
    email = models.EmailField()
    token = models.CharField(max_length=32, default=get_random_string
   (32), editable=False, unique=True)
```

```
29     created_at = models.DateTimeField(auto_now_add=True)
30     TOKEN_VALIDITY_PERIOD = timezone.timedelta(hours=1)
```

Listing 2.1 – school/home_auth/models.py

Analyse ligne par ligne :
Ligne 9 : `class CustomUser(AbstractUser):`
Pourquoi ? Hériter d'`AbstractUser` (plutôt que `AbstractBaseUser`) nous permet d'étendre la structure existante (permissions, hashage de mot de passe natif de Django via PBKDF2) sans avoir à réécrire la logique complète d'authentification from scratch.
Ligne 11 : `email = models.EmailField(unique=True, db_index=True)`
Le paramètre `unique=True` convertit l'email en identifiant principal. L'ajout de `db_index=True` indique à la base SQL de ranger les emails sous forme d'arbre B-tree. Les temps de recherche lors de millions de connexions (Login) sont drastiquement accélérés.
Lignes 18-20 : `is_student, is_teacher, is_admin`
Ces variables booléennes définissent le RBAC (Role-Based Access Control) nativement. Au chargement d'une page (request HTTP), le flag est appelé en mémoire interne.
Ligne 25 : `PasswordResetRequest`
Implémentation d'un modèle éphémère de récupération. On lie ce modèle via une `ForeignKey` à l'utilisateur ciblé, et on insère un jeton cryptographique généré aléatoirement (`get_random_string(32)`) afin qu'il soit inséré dans l'URL de réinitialisation. La `TOKEN_VALIDITY_PERIOD` (1 heure) respecte la norme O-Wasp de sécurité.

2.2 Code Source : 'urls.py' (Logique de Routage)

Le routeur aiguille les requêtes HTTP depuis l'architecture Front-end vers le dispatcher Backend.

```
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('signup/', views.signup_view, name='signup'),
6      path('login/', views.login_view, name='login'),
7      path('logout/', views.logout_view, name='logout'),
8      path('forgot-password/', views.forgot_password, name='forgot-
       password'),
9      path('reset_password/<str:token>/', views.reset_password, name='
       reset_password'),
10 ]
```

Listing 2.2 – school/home_auth/urls.py

Analyse ligne par ligne :
Dans chaque objet `path()` : le premier paramètre définit l'endpoint HTTP cible. Le second paramètre (ex : `views.login_view`) pointe vers la fonction Python exacte dans la RAM.
Le troisième paramètre magique `name='login'` est le nommage dynamique du *Reverse Routeur* de Django. Il permet dans nos templates d'utiliser la balise DTL `{% url 'login' %}` afin que le parseur HTML génère l'URL absolue correspondante ("/login/"). Si demain nous changeons l'adresse à "/connexion/", nul besoin de mettre à jour des centaines de fichiers HTML. La ligne `<str:token>` capture une chaîne de caractère (le jeton lié au PasswordReset) dans l'URL et le passe en argument propre à la vue `views.reset_password(request,`

token).

2.3 Code Source : ‘views.py‘ (L’intelligence Métier MVC)

La vue `signup_view` accomplit le traitement lourd de validation manuelle de profil.

```
1  def signup_view(request):
2      if request.method == 'POST':
3          first_name = request.POST['first_name']
4          last_name = request.POST['last_name']
5          email = request.POST['email']
6          password = request.POST['password']
7
8          # Cr er l'utilisateur
9          user = CustomUser.objects.create_user(
10             username=email, email=email,
11             first_name=first_name, last_name=last_name,
12             password=password
13         )
14
15         # Assigner le r le
16         role = request.POST.get('role', '').lower()
17         if role == 'student':
18             user.is_student = True
19         elif role == 'teacher':
20             user.is_teacher = True
21         elif role == 'admin':
22             user.is_admin = True
23         else:
24             messages.error(request, "Please select a valid role.")
25             return render(request, 'authentication/register.html')
26
27         user.save()
28         login(request, user)
29         messages.success(request, 'Signup successful!')
30         return redirect('index')
31
32     return render(request, 'authentication/register.html')
```

Listing 2.3 – school/home_auth/views.py (Extraits)

Analyse Fonctionnelle :

Ligne 2 : `if request.method == 'POST':`
Il s’agit d’une vue Fonctionnelle. Le navigateur requiert `/signup/`. Si l’utilisateur a cliqué sur un lien classique, c’est un ‘GET‘ (la méthode va directement à `return render()` en bas). Si l’utilisateur clique sur "S’inscrire", la méthode reçue par W.S.G.I est un ‘POST‘ et son Body contient les payload sécurisés.

Ligne 9 : `create_user(...password=password)`
Pourquoi utiliser la méthode propre au Manager ORM `create_user` et non `CustomUser.objects.creat`
Car `create_user` s’assure de hasher le mot de passe (via l’algorithme Pbkdf2_sha256). Autrement, le mot de passe serait rentré "en clair" dans la base SQL.

Ligne 15-20 : Vérification de la payload "role". Une valeur cachée ‘SELECT‘ est retournée et assigne le booléen RBAC correspondant via un branchement conditionnel binaire (‘if / elif‘).

Ligne 27 : `user.save()`

Force le contexte Python à s'exécuter en un commit SQL (`INSERT INTO...`).

2.4 Les Interfaces Utilisateurs : 'register.html' et 'login.html'

Plongeons nos regards sur les pages d'entrée des utilisateurs. Les requêtes y sont formatées et renvoyées à Django.

```
1  <form method="POST" action="{% url 'signup' %}">
2      {% csrf_token %}
3      <div class="form-group">
4          <input class="form-control" type="text" name="first_name"
       placeholder="First Name" required>
5      </div>
6      <div class="form-group">
7          <select class="form-control" name="role" required>
8              <option value="">Select Role</option>
9              <option value="student">Student</option>
10             <option value="teacher">Teacher</option>
11             <option value="admin">Admin</option>
12         </select>
13     </div>
14     <div class="form-group mb-0">
15         <button class="btn btn-primary btn-block" type="submit">Register
       </button>
16     </div>
17 </form>
```

Listing 2.4 – school/templates/authentication/register.html (Analyse)

Ligne 1 : `<form method="POST" action="{% url 'signup' %}">`
La balise DTL `{% url 'signup' %}` est exécutée par le moteur Django côté Backend, qui recherche l'url nommée "signup" et remplace cette chaîne par "/signup/". Cette étape se déroule avant même que le fichier HTML soit envoyé au client (Server-Side Rendering).
Ligne 2 : `{% csrf_token %}`
Cette ligne magique est une exigence absolue de sécurité en Django. Elle prévient les attaques **Cross-Site Request Forgery** en créant un jeton temporaire alphanumérique (parfois appelé nonce) qui vérifie en interne si le formulaire provient réellement du site web d'origine et non d'un acteur malveillant ayant aspiré le DOM.
UI Bootstrap : Les classes `form-group`, `form-control`, `btn`, `btn-primary` et `btn-block` injectent le style CSS du framework externe Bootstrap qui gère la flexibilité de l'affichage mobile. En insérant `name="role"`, ce paramètre voyage dans l'en-tête du Header HTTPS, pour être exploité ligne 16 de notre fichier `views.py` de la partie précédente : `request.POST.get('role')` ; bouclant ainsi la boucle logicielle.

Chapitre 3

L'Application faculty : Structuration Pédagogique

Au cœur de toute institution académique se trouve la structuration de son savoir. L'application `faculty` a été pensée pour cartographier physiquement et virtuellement les Départements (Facultés) ainsi que les Matières (Subjects) qui y sont enseignées. Ce chapitre explique le socle M.V.T (Model-View-Template) utilisé pour générer un C.R.U.D complet basé sur les classes (*Class-Based Views*) de Django, limitant les redondances de code.

3.1 Modélisation de Base de Données (models.py)

La structure relationnelle est ici indispensable pour lier les professeurs aux départements et les matières à ces derniers.

```python
from django.db import models
from teacher.models import Teacher

class Department(models.Model):
    name = models.CharField(max_length=150)
    head_of_department = models.ForeignKey(Teacher, on_delete=models.SET_NULL, null=True, blank=True, related_name='headed_departments')
    description = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name

class Subject(models.Model):
    name = models.CharField(max_length=150)
    code = models.CharField(max_length=50, unique=True)
    department = models.ForeignKey(Department, on_delete=models.CASCADE, related_name='subjects')
    teacher = models.ForeignKey(Teacher, on_delete=models.SET_NULL, null=True, blank=True, related_name='subjects')

    def __str__(self):
        return f"{self.name} ({self.code})"
```

Listing 3.1 – school/faculty/models.py

Analyse technique et conceptuelle ligne par ligne :
Ligne 6 : `head_of_department = models.ForeignKey(Teacher, on_delete=models.SET_NULL, null=True, ...)`
1. *Le Choix de la Clé Étrangère* : Un département n'a qu'un seul Chef, mais un professeur pourrait hypothétiquement être chef de plusieurs petits départements, d'où la relation unidirectionnelle 'One-to-Many' (ForeignKey) au lieu de 'OneToOne'.
2. *Le comportement de suppression* : `on_delete=models.SET_NULL` est capital. Dans un système d'information réel, si un professeur démissionne et que son instance `Teacher` est supprimée, la règle `CASCADE` aurait provoqué la destruction pure et simple du Département d'Informatique. Avec `SET_NULL`, le département reste fonctionnel, mais perd son chef (le champ devient NULL).
Ligne 17 : `department = models.ForeignKey(..., related_name='subjects')`
Le paramètre 'related$_name$'estunReverse Accessorextrmementpuissantdel'ORMDjango.Sanslui, pour `Subject.objects.filter(department=dept_info)`.Grceau'related$_name$', nouspouvonsutiliserlanota objetdirecte : `dept_info.subjects.all()`.

3.2 La Logique d'Affichage : Vues Basées sur les Classes (views.py)

Afin de respecter le principe DRY (*Don't Repeat Yourself*), nous avons repoussé les vues fonctionnelles classiques au profit des C.B.V (Class-Based Views).

```
1  from django.urls import reverse_lazy
2  from django.views.generic import ListView, CreateView, UpdateView,
       DeleteView
3  from .models import Department
4  from django.contrib.auth.mixins import LoginRequiredMixin,
       UserPassesTestMixin
5
6  class AdminRequiredMixin(UserPassesTestMixin):
7      def test_func(self):
8          return self.request.user.is_authenticated and getattr(self.
       request.user, 'is_admin', False)
9
10 class DepartmentListView(LoginRequiredMixin, ListView):
11     model = Department
12     template_name = 'faculty/department_list.html'
13     context_object_name = 'department_list'
14
15 class DepartmentCreateView(AdminRequiredMixin, CreateView):
16     model = Department
17     form_class = DepartmentForm
18     template_name = 'faculty/department_form.html'
19     success_url = reverse_lazy('department-list')
20
21 class DepartmentDeleteView(AdminRequiredMixin, DeleteView):
22     model = Department
23     template_name = 'faculty/department_confirm_delete.html'
24     success_url = reverse_lazy('department-list')
```

Listing 3.2 – school/faculty/views.py (Extraits des Vues Department)

Analyse technique ligne par ligne :
Lignes 6-8 : `AdminRequiredMixin(UserPassesTestMixin)`

Ce *Mixin* surchargé agit comme une barrière de péage. La fonction `test_func()` vérifie que la personne connectée possède un jeton `is_admin = True`. Si un élève tente de forcer l'URL de création de département, ce Mixin déclenchera une erreur 403 Forbidden avant même l'exécution locale de la vue.
Lignes 10-13 : `class DepartmentListView(LoginRequiredMixin, ListView)`
En héritant de 'ListView', Django comprend implicitement qu'il doit exécuter la requête ORM : `Department.objects.all()`. Le résultat de cette requête SQL 'SELECT * FROM faculty$_{d}epartment$'$estempaquetdynamiquementdanslavariablelocale$`department_list`$avantd'treenvo$
Lignes 15-19 : `class DepartmentCreateView`
$CettevuegrelesdeuxspectresduprotocoleHTTP.Enmthode$'$GET$', $elleinstancievide$`DepartmentForme`tl
$1.Extraction$: '$form = DepartmentForm(request.POST)$'$2.Scurit$: '$iffform.is_{v}alid()$:
'$3.Persistance$: '$form.save()$'$(quignreun$'$INSERTINTO$'$enbase).Alacompltionalgorithmique, sisuc$

3.3 Aiguillage et Variables (urls.py)

Les URLs utilisent les *kwargs* (Keyword Arguments) pour passer les identifiants clés.

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     # Department URLs
6     path('departments/', views.DepartmentListView.as_view(), name='
    department-list'),
7     path('departments/add/', views.DepartmentCreateView.as_view(), name=
    'department-add'),
8     path('departments/<int:pk>/edit/', views.DepartmentUpdateView.
    as_view(), name='department-edit'),
9     path('departments/<int:pk>/delete/', views.DepartmentDeleteView.
    as_view(), name='department-delete'),
10 ]
```

Listing 3.3 – school/faculty/urls.py

La ligne critique : `<int:pk>`
Lorsqu'un administrateur souhaite modifier le département "Sciences" (qui porte l'Identifiant unique 5 dans la RAM), son navigateur appelle l'URL '/departments/5/edit/'. L'expression `<int:pk>` capture le numéro 5, s'assure qu'il s'agit bien d'un Entier ('int'), et le passe au paramètre Primary Key ('pk') de l'UpdateView.

3.4 L'Interface et le Langage de Template (HTML)

Nous analysons ici le fichier servant à visualiser les départements sous forme de table interactive Bootstrap.

```
1 {% extends 'Home/base.html' %}
2 {% block body %}
3     <tbody>
4         {% for item in department_list %}
5         <tr>
6             <td>{{ item.id }}</td>
7             <td><h2><a>{{ item.name }}</a></h2></td>
8             <td>{{ item.head_of_department|default:"-" }}</td>
```

```
9            <td>{{ item.created_at|date:"Y" }}</td>
10           <td class="text-right">
11               <div class="actions">
12                   <a href="{% url 'department-edit' item.id %}" class=
    "btn btn-sm bg-success-light">
13                       <i class="fas fa-pen"></i>
14                   </a>
15
16                   <form action="{% url 'department-delete' item.id %}"
     method="POST" style="display:inline;">
17                       {% csrf_token %}
18                       <button type="submit" class="btn btn-sm bg-
    danger-light" onclick="return confirm('Etes-vous sur de vouloir
    supprimer {{ item.name }} ?');">
19                           <i class="fas fa-trash"></i>
20                       </button>
21                   </form>
22               </div>
23           </td>
24       </tr>
25       {% empty %}
26       <tr>
27           <td colspan="5" class="text-center">No Departments Found</td
    >
28       </tr>
29       {% endfor %}
30   </tbody>
31 {% endblock %}
```

Listing 3.4 – school/faculty/templates/faculty/department_list.html (Extrait de la Boucle)

Analyse des Balises DTL (Django Template Language) ligne par ligne :

Ligne 1 : `{% extends 'Home/base.html' %}`

C'est la pierre angulaire de DRY en frontend. Ce fichier HTML ne génère pas de '<html>', '<head>' ou '<body>'. Il hérite virtuellement de toute la charge graphique (Sidebar, Navbar, CSS Bootstrap) de 'base.html' et injecte ses données via `{% block body %}`.

Ligne 5 : `{% for item in department_list %}`

Le moteur initie une itération logique sur le QuerySet SQL ramené par le backend. Pour chaque ligne de la base, Django dupliquera la portion HTML '<tr...>' jusqu'au '**Ligne 7** : `{{ item.name }}`

L'usage de double accolades indique une variable ou une méthode de classe à imprimer. Le moteur effectue un '$_{str}$ $_{ourcuprelaStringduchamp'name'}$.**Ligne 9**:`{{ item.head_of_department|default:"-" }}`

Chapitre 4

La Gestion des Acteurs : Les Applications student et teacher

La force d'un système d'information de gestion scolaire (SIGS) réside dans sa capacité à traiter des identités complexes. Contrairement à une simple application d'e-commerce, un "Étudiant" n'est pas qu'un client : c'est une entité mineure subordonnée à une entité juridique tutélaire (le Parent). Ce chapitre dissèque comment l'application gère cette imbrication au niveau de la base de données, des vues et de l'interface utilisateur.

4.1 Modélisation de la Dépendance (models.py)

La conception de la base de données sépare virtuellement l'Étudiant de son Tuteur pour normaliser les données, tout en les liant par une relation matricielle absolue.

```
from django.db import models

class Parent(models.Model):
    father_name = models.CharField(max_length=100)
    father_occupation = models.CharField(max_length=100, blank=True)
    father_mobile = models.CharField(max_length=15)
    father_email = models.EmailField(max_length=100)
    mother_name = models.CharField(max_length=100)
    mother_mobile = models.CharField(max_length=15)
    present_address = models.TextField()

class Student(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    student_id = models.CharField(max_length=20)
    gender = models.CharField(max_length=10, choices=[('Male', 'Male'),
   ('Female', 'Female')])
    date_of_birth = models.DateField()
    student_class = models.CharField(max_length=50)
    student_image = models.ImageField(upload_to='students/', blank=True)

    # La relation vitale
    parent = models.OneToOneField(Parent, on_delete=models.CASCADE)

    def __str__(self):
```

```
25        return f"{self.first_name} {self.last_name} ({self.student_id})"
```

Listing 4.1 – school/student/models.py

Analyse Conceptuelle et Ligne par Ligne :
Ligne 19 : `student_image = models.ImageField(upload_to='students/', blank=True)`
Le framework Django ne stocke jamais les images sous forme de données binaires (BLOB) dans la base SQL pour des raisons de performance. À la place, 'ImageField' sauvegarde physiquement le fichier dans un répertoire serveur ('media/students/') et n'inscrit dans la base de données que le chemin (String) vers ce fichier. L'attribut 'blank=True' autorise un étudiant à ne pas avoir d'avatar à son inscription.
Ligne 22 : `parent = models.OneToOneField(Parent, on_delete=models.CASCADE)`
C'est le sommet de l'ingénierie relationnelle pour cette application. La cardinalité `OneToOne` (1 :1) agit comme une contrainte 'UNIQUE' sur une 'ForeignKey'. Cela garantit mathématiquement qu'un objet `Parent` ne peut être lié qu'à un seul et unique enregistrement `Student` dans ce contexte scolaire (et inversement). Si l'institutrice supprime le profil de l'étudiant, la directive `on_delete=models.CASCADE` va déclencher un "Trigger" SQL interne qui effacera immédiatement les données obsolètes du Parent de la base de données, évitant ainsi le stockage fantôme de données personnelles.

4.2 L'Intelligence Multi-Modèle : La Logique des Vues (views.py)

Comment un formulaire HTML unique peut-il nourrir trois tables SQL différentes (Parent, Student, CustomUser) ? C'est le rôle du contrôleur `add_student`.

```
1  @admin_required
2  def add_student(request):
3      if request.method == 'POST':
4          # (1) Extraction ommise pour brevet  : request.POST.get('...')
5          # request.FILES.get('student_image') g re le flux binaire d'
   images
6
7          # 1. Cr er et sauvegarder l'entit  Parent EN PREMIER
8          parent = Parent.objects.create(
9              father_name=father_name,
10             father_mobile=father_mobile,
11             mother_name=mother_name,
12             present_address=present_address
13         )
14
15         # 2. Cr er l'entit   tudiant   li e au Parent
16         Student.objects.create(
17             first_name=first_name,
18             last_name=last_name,
19             student_id=student_id,
20             student_image=student_image,
21             parent=parent # <- Injection de l'instance sauvegard e
   pr c demment
22         )
23
24         # 3. G n ration du Compte Utilisateur (Login)
25         if not CustomUser.objects.filter(username=student_id).exists():
26             CustomUser.objects.create_user(
```

```
27                 username=student_id,
28                 email=f"{student_id}@student.preskool.com",
29                 password=student_id,
30                 is_student=True,
31                 first_name=first_name,
32                 last_name=last_name
33             )
34
35         messages.success(request, 'Student added Successfully')
36         return redirect('student_list')
37     else:
38         return render(request, 'students/add-student.html')
```

Listing 4.2 – school/student/views.py (Extrait de la méthode add_student)

Explication Ligne par Ligne du "Pourquoi" :
Ligne 8 - Ligne 16 : Pourquoi créer le ‘Parent‘ avant le ‘Student‘ ? C’est une obligation imposée par le moteur relationnel (SGBD). Le ‘Student‘ requiert une ‘parent$_id$‘$(Cltechnique).Tantquel'i$
Ligne 21 : `parent=parent`
$L'ORMdeDjangolielgammentl'instance.Iltraduiracelapar‘parent_id = 5‘danslarequte‘INSERTINT$
Lignes 25-33 : **L’Automatisation Ingénieuse.**
$Pluttqued'obligerl'administrationmatriculairecrerunprofillve, puisallerdansl'interfacedescuritgnre$

4.3 Analyse des Templates HTML de Profil et de Listing

L’interface doit maintenant afficher ces données complexes et ces jointures SQL sur l’écran du principal.

```
1  {% extends 'Home/base.html' %}
2  {% load static %}
3  {% block body %}
4      <tbody>
5          {% for student in student_list %}
6          <tr>
7              <td>{{ student.student_id }}</td>
8              <td>
9                  <h2 class="table-avatar">
10                     <a href="{% url 'view_student' student_id=student.id
   %}">
11                         {{ student.first_name }} {{ student.last_name }}
12                     </a>
13                 </h2>
14             </td>
15             <td>{{ student.date_of_birth|date:"d M Y" }}</td>
16             <td>{{ student.parent.father_name }} / {{ student.parent.
   mother_name }}</td>
17
18             <td class="text-right">
19                 <div class="actions">
20                     <form action="{% url 'delete_student' student_id=
   student.id %}" method="POST" style="display:inline;">
21                         {% csrf_token %}
22                         <button type="submit" class="btn btn-sm bg-
   danger-light"
```

```
23                                  onclick="return confirm('Are you sure
    you want to delete this student?');">
24                                <i class="fas fa-trash"></i>
25                            </button>
26                        </form>
27                    </div>
28                </td>
29            </tr>
30            {% endfor %}
31        </tbody>
32
33        <script src="{% static 'assets/js/jquery-3.6.0.min.js' %}"></script>
34        <script src="{% static 'assets/plugins/datatables/datatables.min.js'
    %}"></script>
35    {% endblock %}
```

Listing 4.3 – school/templates/students/students.html

Analyse Rigoureuse des Appels (DTL) :

Ligne 2 : `{% load static %}`
Cette instruction alerte le moteur de rendu Django qu'il doit importer le processeur de fichiers statiques (Images abstraites, CSS, JS). C'est crucial pour l'architecture en production.

Ligne 17 : `{{ student.parent.father_name }}`
C'est ici qu'intervient la prouesse de l'ORM Django, appelée **"Forward Traversal"** (Traversée en avant). Dans le fichier HTML, en utilisant de simples "points", nous ordonnons algorithmiquement à Django de faire un 'LEFT JOIN' dynamique depuis l'itération actuelle de la table 'Student' vers la table 'Parent' pour en extirper la colonne 'father$_name$'.$Ledveloppeurn'apaseucrireuneseulelignedeSQLcomplexedanssavue$!

Ligne 36 et 37 : `src="{% static 'assets/...min.js' %}"`
$Enproduction, lesfichiersd'effetsvisuelsetdestructurecommejQueryouDataTables(pourtrierdynamiq$
$Fichierlocalnontrouv"$.

Chapitre 5

Modules Avancés : Planification, Examens et Résultats

Au-delà de la stricte gestion d'identités, un établissement scolaire nécessite une horlogerie précise pour fonctionner. Les modules "Planning" (pour les Jours Fériés et Emplois du temps) et "Exams" constituent le cœur de la machinerie éducative de PreSkool. Ce cinquième chapitre dévoile l'architecture avancée de la modélisation sous contraintes mathématiques SQL et les filtres de sécurité restrictifs appliqués aux Vues.

5.1 Modélisation de la Planification Temporelle (planning/models.py)

Les entités liées au temps sont par nature volatiles. L'emploi du temps permet la projection d'un espace-temps (Jour/Heure) vers la ressource humaine (Professeur) et cognitive (Matière).

```
from django.db import models
from faculty.models import Subject
from teacher.models import Teacher

class Holiday(models.Model):
    title = models.CharField(max_length=100)
    start_date = models.DateField()
    end_date = models.DateField()
    description = models.CharField(max_length=300)

    def __str__(self):
        return self.title

class TimeTable(models.Model):
    DAY_CHOICES = [
        ('Monday', 'Monday'),
        ('Tuesday', 'Tuesday'),
        ('Wednesday', 'Wednesday'),
        ('Thursday', 'Thursday'),
        ('Friday', 'Friday'),
        ('Saturday', 'Saturday'),
        ('Sunday', 'Sunday'),
    ]

```

```
25     day = models.CharField(max_length=15, choices=DAY_CHOICES)
26     start_time = models.TimeField()
27     end_time = models.TimeField()
28     subject = models.ForeignKey(Subject, on_delete=models.CASCADE)
29     teacher = models.ForeignKey(Teacher, on_delete=models.SET_NULL, null
    =True, blank=True)
30     student_class = models.CharField(max_length=50) # Matching target
31
32     def __str__(self):
33         return f"{self.student_class} - {self.subject.name} ({self.day})
    "
```

Listing 5.1 – school/planning/models.py

Analyse Décisionnelle Ligne par Ligne :
Lignes 16-24 : L'utilisation d'une énumération native `DAY_CHOICES` agit comme une contrainte 'CHECK' en SQL. Elle interdit mathématiquement l'insertion d'un jour erroné (ex : "Lundi" ou "Funday") dans la base de données, garantissant la standardisation ISO des requêtes d'agenda.
Ligne 30 : `teacher = models.ForeignKey(..., on_delete=models.SET_NULL)`
Encore une fois, l'architecture prime sur la simplicité. Si le professeur d'Histoire géographie tombe gravement malade et/ou est supprimé de la base, on ne doit surtout pas déclencher un `CASCADE` qui effacerait le créneau du Lundi de 8h à 10h. Le 'SET$_NULL'indiquel'administrationqu'ily$

5.2 Modélisation Évaluative et Contraintes de Multiplicité (exam/models.py)

Comment garantir, à grande échelle, qu'un étudiant ne reçoive pas deux bulletins de notes pour le même examen ou la même matière évaluée ?

```
1  from django.db import models
2  from faculty.models import Subject
3  from student.models import Student
4
5  class Exam(models.Model):
6      title = models.CharField(max_length=150)
7      date = models.DateField()
8      subject = models.ForeignKey(Subject, on_delete=models.CASCADE,
    related_name='exams')
9      description = models.TextField(blank=True)
10
11 class Result(models.Model):
12     exam = models.ForeignKey(Exam, on_delete=models.CASCADE,
    related_name='results')
13     student = models.ForeignKey(Student, on_delete=models.CASCADE,
    related_name='results')
14     marks_obtained = models.DecimalField(max_digits=5, decimal_places=2)
15     total_marks = models.DecimalField(max_digits=5, decimal_places=2,
    default=20.00)
16     remarks = models.CharField(max_length=200, blank=True)
17
18     class Meta:
19         unique_together = ('exam', 'student')
20
21     def __str__(self):
```

```
22         return f"{self.student.first_name} {self.student.last_name} - {
    self.exam.title}"
```

Listing 5.2 – school/exam/models.py

Analyse Critique de l'Architecture des Notes :
Ligne 19 : `marks_obtained = models.DecimalField(max_digits=5, decimal_places=2)`
Stocker des notes en 'FloatField' en Python soulève l'irrégularité de l'arithmétique à virgule flottante IEEE 754 (ex : '19.5 + 0.1 = 19.600000000000001'). Le composant `DecimalField` utilise l'algèbre exacte, cruciale pour les calculs de moyennes lors du passage d'année.
Ligne 24 : **L'ingénierie Magistrale :** `unique_together = ('exam', 'student')`
C'est le point d'orgue de cette application. Dans la sous-classe de paramétrage `Meta`, nous imposons au moteur relationnel (SQLite ou PostgreSQL) de créer un "Index Unique Composé" sur l'axe multidimensionnel '(ID$_Examen, ID_Etudiant)$'.$Sil'enseignant, paraccident, validedeuxf$

5.3 Cybersécurité et Partitionnement Dynamique des Vues (exam/views.py)

Restreindre la lecture des données sensibles (loi RGPD). Un étudiant ne doit pas pouvoir espionner les notes des autres.

```
1  from django.views.generic import ListView
2  from .models import Result
3  from django.contrib.auth.mixins import LoginRequiredMixin
4
5  class ResultListView(LoginRequiredMixin, ListView):
6      model = Result
7      template_name = 'exam/result_list.html'
8      context_object_name = 'result_list'
9
10     def get_queryset(self):
11         user = self.request.user
12         val_is_student = getattr(user, 'is_student', False)
13
14         # Assumption: username = student_id
15         if val_is_student:
16             return Result.objects.filter(student__student_id=user.
    username)
17         return Result.objects.all()
```

Listing 5.3 – school/exam/views.py

L'Audit Technologique Ligne par Ligne :
Ligne 10 : La surcharge de `def get_queryset(self)`.
D'ordinaire, `ListView` renvoie machinalement toute la table de la base de données ('SELECT * FROM exam$_result$').$Surchargercettemthodenouspermetdeplacercequ'onnommeun"FiltredeS$
Lignes 11-12 : `user = self.request.user`$extraitl'identitdesessionactivedelapileRAM.$
Lignes 15-16 : `if val_is_student : return Result.objects.filter(student__student_id=use`
$Cetteseuleligneralisel'exploitdescuriserdesmilliersdelignesclassifies.Ledoubleunderscore('tudiant_{st}$

5.4 L'Hybridation Dynamique du Template (exam/result_list.html)

Le frontend ne possède aucune capacité de restriction (il est piloté par l'utilisateur). Toutefois, il doit être suffisamment intelligent pour cacher dynamiquement l'interface d'édition.

```
1  {% extends 'Home/base.html' %}
2  {% block body %}
3              <div class="col-auto text-right float-right ml-auto">
4                  {% if request.user.is_admin or request.user.is_teacher
   %}
5                      <a href="{% url 'result-add' %}" class="btn btn-
   primary">
6                          <i class="fas fa-plus"></i> Saisir une Note
7                      </a>
8                  {% endif %}
9              </div>
10
11             <tbody>
12                 {% for item in result_list %}
13                 <tr>
14                     <td>{{ item.student.first_name }} {{ item.student.
   last_name }}</td>
15                     <td>{{ item.exam.title }}</td>
16                     <td>
17                         <h2>{{ item.marks_obtained }} / {{ item.
   total_marks }}</h2>
18                     </td>
19                     <td>
20                         <span class="badge badge-{% if item.
   marks_obtained >= 10 %}success{% else %}danger{% endif %}">
21                             {% if item.marks_obtained >= 10 %}Pass{%
   else %}Fail{% endif %}
22                         </span>
23                     </td>
24                     {% if request.user.is_admin or request.user.
   is_teacher %}
25                         <td class="text-right">
26                            <a href="{% url 'result-edit' item.id %}"
   class="btn btn-sm bg-success-light">
27                                <i class="fas fa-pen"></i> Modifier
28                            </a>
29                         </td>
30                     {% endif %}
31                 </tr>
32                 {% endfor %}
33             </tbody>
34 {% endblock %}
```

Listing 5.4 – school/templates/exam/result_list.html (Extrait UI)

Radiographie des Décisions Front-End DTL :
Lignes 4-8 & 26-30 : Balise conditionnelle `{% if request.user.is_admin... %}`
L'objet 'request' est globalement injecté dans les contextes des Templates Django. Cette ligne effectue une vérification booléenne à la volée. Si l'étudiant est connecté, le bouton "Saisir une Note" et la colonne entière d'actions "Modifier" disparaissent purement et

simplement du DOM. Le navigateur web de l'étudiant ne saura même jamais que ces endpoints d'accès existent.

Ligne 22 : `{% if item.marks_obtained >= 10 %}`

Le moteur DTL ne se contente pas d'imprimer des variables ; il possède des capacités de logiques abstraites. Ce fragment conditionnel vérifie dynamiquement si le résultat de la requête ORM (en float implicite) atteint la moyenne de '10'. Si oui, la classe visuelle Bootstrap 'badge-success' est greffée en temps réel au rendu HTML, affichant un encart vert rassurant. La classe 'badge-danger' (rouge cramoisi) peindra l'écran des élèves échouant, permettant un suivi pédagogique optique instantané.

Chapitre 6

Architecture Frontend et Bilan d'Ingénierie

Dans les chapitres précédents, nous avons consolidé les fondations arrières du système (Backend et Base de Données). Ce dernier chapitre se penche sur le point de contact final avec l'humain : l'Interface Utilisateur (Frontend). Nous y détaillerons l'anatomie du Layout principal, avant de dresser un bilan critique des défis relevés et des jalons futurs pour le projet PreSkool.

6.1 L'Héritage Dynamique (base.html)

Plutôt que de dupliquer les en-têtes HTML sur chaque page, l'architecture DTL (Django Template Language) utilise un patron de conception appelé "Template Inheritance". `base.html` est le squelette maître du projet.

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
   <head>
      <meta charset="utf-8">
      <meta name="viewport" content="width=device-width, initial-scale
   =1.0, user-scalable=0">
      <title>Preskool - Dashboard</title>

      <link rel="stylesheet" href="https://fonts.googleapis.com/css2?
   family=Poppins:ital,wght@0,500;0,600;0,700;1,400&amp;display=swap">
      <link rel="stylesheet" href="{% static 'assets/plugins/bootstrap/
   css/bootstrap.min.css' %}">
      <link rel="stylesheet" href="{% static 'assets/plugins/fontawesome
   /css/all.min.css' %}">
      <link rel="stylesheet" href="{% static 'assets/css/style.css' %}">
   </head>
   <body>
      <div class="main-wrapper">
         <div class="sidebar" id="sidebar">
            <div class="sidebar-inner slimscroll">
               <div id="sidebar-menu" class="sidebar-menu">
                  <ul>
                     <li class="menu-title"><span>Main Menu</span></li>
                     <li class="submenu active">
```

```
22                          <a href="#"><i class="fas fa-user-graduate"></i>
     <span> Dashboard</span></a>
23                          <ul>
24                             {% if request.user.is_admin %}
25                             <li><a href="{% url 'admin_dashboard' %}">
    Admin Dashboard</a></li>
26                             {% elif request.user.is_teacher %}
27                             <li><a href="{% url 'teacher_dashboard' %}">
    Teacher Dashboard</a></li>
28                             {% elif request.user.is_student %}
29                             <li><a href="{% url 'student_dashboard' %}">
    Student Dashboard</a></li>
30                             {% endif %}
31                          </ul>
32                       </li>
33
34                       {% if request.user.is_admin %}
35                       <li class="submenu">
36                          <a href="#"><i class="fas fa-building"></i> <
    span> Departments</span></a>
37                          <ul>
38                             <li><a href="{% url 'department-list' %}">
    Department List</a></li>
39                             <li><a href="{% url 'department-add' %}">
    Department Add</a></li>
40                          </ul>
41                       </li>
42                       {% endif %}
43                    </ul>
44                 </div>
45              </div>
46           </div>
47
48           {% block body %}
49
50           {% endblock %}
51        </div>
52
53        <script src="{% static 'assets/js/jquery-3.6.0.min.js' %}"></
    script>
54        <script src="{% static 'assets/plugins/bootstrap/js/bootstrap.min.
    js' %}"></script>
55     </body>
56  </html>
```

Listing 6.1 – school/templates/Home/base.html (Extrait de la structure maître)

Autopsie du Squelette Moteur Ligne par Ligne :

Ligne 1 : `{% load static %}`
Obligatoire en début de fichier. Elle ordonne au "Template Engine" de résoudre dynamiquement l'emplacement absolu des fichiers bruts servis par le serveur web, préparant le terrain pour les balises qui suivront.

Lignes 11-13 : `href="{% static 'assets/.../bootstrap.min.css' %}"`
Intégration du framework **Bootstrap CSS**. Grâce à cela, le système hérite d'un "Grid System" mathématiquement divisé en 12 colonnes, rendant l'application PreSkool 100% Responsive (adaptable des écrans d'ordinateurs jusqu'aux smartphones). L'importation de *FontAwesome* livre une vaste bibliothèque vectorielle d'icônes.

Lignes 28-35 : L'Arbre de Décision Polymorphe `request.user`
L'objet `request` est omniprésent en session. Dès que l'utilisateur se connecte, la Sidebar (Barre latérale de gauche) mute. Les conditions `elif` redessinent la topologie des menus cliquables. L'élève ('is$_{s}tudent$')$neverradanssontableaudebordquecequiluiestd.L'administrationverralel$

Lignes 52-54 : $Letrounoir${% `block body` %}
$C'estlepointd'injectionprincipal.Touslesautresfichiersduprojetdel'application(ex$: '$student_{l}ist.html$'

6.2 Défis Techniques d'Ingénierie

Durant la phase de développement et de cycle de vie logiciel, plusieurs obstacles académiques et techniques ont nécessité des corrections drastiques.

6.2.1 1. L'Isolement des Dépendances (Environnements Virtuels)

La première difficulté liée au framework Django réside dans les conflits globaux de paquets locaux. L'installation de bibliothèques directement sur la machine "Hôte" provoque des instabilités (ex : Pillow pour les images ou Psycopg2 pour la BDD). L'instauration radicale d'un *Virtual Environment* ('python -m venv env') a permis d'isoler les versions binaires (pip) garantissant la pérennité du système peu importe l'ordinateur où le dépôt Git est cloné.

6.2.2 2. Héritage des Migrations de Base de Données

Changer la structure fondamentale de Django (`AbstractUser` → `CustomUser`) *après* avoir exécuté les commandes `makemigrations` et `migrate` natales a provoqué une erreur fatale dite de *Inconsistent Migration History*. Le moteur ORM refusait d'écrire sur la table 'auth$_{u}ser'standard.Lasolutionarchitecturalefutdedtruirercursivementlefichierd'tatbinairelocal('db.$

6.2.3 3. Fichiers Statiques et Serveur de Production

Si Django sert très bien les fichiers statiques ('/static/...') avec sa variable interne `DEBUG = True`, en mode "Production", ce mécanisme coupe la communication pour des raisons de sécurité. La structuration a nécessité la mise en place de variables `STATIC_ROOT` et `STATICFILES_DIRS` afin de compacter numériquement le dossier `assets` lors de la commande `collectstatic`.

6.3 Pistes d'Améliorations et Perspectives (Évolutivité)

PreSkool, dans sa forme actuelle, est un *Minimum Viable Product* (MVP) extrêmement robuste. Néanmoins, l'architecture s'ouvre à des itérations futures appelées "Scalabilité".

— **Création d'une API RESTful (Django Rest Framework)** : L'ensemble du code Backend actuel est fortement couplé au Frontend DTL. En implémentant *DRF*, nous pourrons sérialiser les modèles (`Student`, `Result`) en format JSON. Cette mutation permettrait le développement natif d'une **Application Mobile** (ex : React Native ou Flutter) pour les téléphones des élèves et parents.

— **Microservices de Transaction (Paiement)** : L'un des piliers manquants est l'entité comptable. Une future passe d'ingénierie intégrerait les API sécurisées telles que Stripe ou PayPal, offrant à la vue 'Parent' la capacité de finaliser les frais de scolarité via un terminal *Webhooks* asynchrone.
— **Déploiement Cloud Industrialisé** : L'utilisation d'une base SQLite est inadaptée à de multiples transactions concurrentes. Un futur axe prioritaire sera la "Dockerisation" du système. L'application PreSkool serait empaquetée dans un conteneur configuré avec un serveur WSGI professionnel (Gunicorn) interconnecté au proxy Nginx, opérant les données sur un *Cluster* PostgreSQL managé sur AWS ou Azure.

6.4 Conclusion Générale

Le projet **PreSkool** a prouvé que la création d'un système de gestion éducative (SIGS) transcende le simple stockage de données. C'est un exercice profond de modélisation algorithmique et d'intégrité relationnelle. À travers l'interface M.V.T de Django, nous avons construit un monde virtuel étanche où la Sécurité (Authentification / RBAC) est régie par des Vues et des Mixins infaillibles, où la cohérence des examens est bridée par des index algébriques complexes ('unique$_{t}ogether$'), $etolacommunicationasymtrique(ProfesseurvsMoteurdeRend$

– FIN DU RAPPORT ARCHITECTURAL –